

资源约束下高层次综合列表调度算法的优化实现与评估

Implementation and Evaluation of Resource-Constrained

List Scheduling in HLS

姓名：陈润锴

学号：23362012

课程：数据结构

日期：2026 年 6 月

摘要

集成电路规模越来越大，传统 RTL 级手写设计效率跟不上芯片复杂度增长的速度。高层次综合（HLS）可以把代码描述的行为级算法自动转成硬件电路，其中调度（Scheduling）这一步直接决定了最后电路的性能和面积。本项目主要实现了资源约束下的列表调度（List Scheduling）算法，实现了完整的静态时序分析

（ASAP/ALAP）、基于迁移力（Mobility）的优先级排序策略以及资源受限的动态任务分配引擎。同时，项目构建了小、中、大三级规模的测试数据集，设计了策略对比实验、灵敏度分析实验、算法复杂度验证实验与可调度性分析实验，并开发了甘特图、资源利用率曲线等可视化工具。实验结果表明，基于 Mobility 最小优先的启发式策略在中等规模 FIR 滤波器数据集上相较于随机策略能够降低 20.4% 的调度延迟，验证了本算法的有效性与工程实用价值。

Abstract

With the continuous expansion of integrated circuit scale, traditional RTL-level design efficiency can no longer meet the demands of increasingly complex chip development. High-Level Synthesis (HLS) technology can automatically convert algorithms described in high-level languages into hardware circuits, where the scheduling step directly determines the final performance and area of the chip. This project focuses on resource-constrained List Scheduling algorithms, implementing complete static timing analysis (ASAP/ALAP), Mobility-based priority sorting strategies, and a dynamic task allocation engine under resource constraints. Meanwhile, the project constructs small, medium, and large-scale test datasets, designs strategy comparison experiments, sensitivity analysis experiments, scalability tests and schedulability tests, and develops visualization tools such as Gantt charts (with critical path highlighting) and resource utilization curves. Experimental results show that the Mobility-minimum-first heuristic strategy can reduce scheduling latency by 20.4% compared to the random strategy on medium-scale FIR filter datasets, verifying the effectiveness and practical value of the algorithm.

目录

摘要 2

Abstract..... 2

目录 3

1. 项目目标与研究背景..... 5

 1.1 HLS 与调度问题概述..... 5

 1.2 资源约束调度的挑战..... 5

 1.3 项目目标..... 5

2. 研究内容与技术路线..... 6

 2.1 数据流图（DFG）建模..... 6

 2.2 ASAP/ALAP 静态时序分析与关键路径提取..... 6

 2.3 List Scheduling 核心算法..... 6

 2.4 优先级策略与资源分配机制..... 7

3. 系统实现..... 7

 3.1 开发环境与依赖..... 7

 3.2 系统架构与模块设计..... 8

 3.3 核心代码解析..... 8

4. 实验设计与测试分析..... 9

 4.1 测试数据集设计..... 9

 4.2 功能验证实验..... 9

 4.3 策略对比实验..... 10

 4.4 灵敏度分析实验..... 11

 4.5 算法复杂度验证实验..... 11

 4.6 可调度性分析实验..... 12

 4.7 实验结果汇总..... 13

4.8 资源绑定 (Binding) 实验.....	14
4.9 面积-延迟帕累托优化分析.....	15
4.10 流水线调度实验.....	17
5. 实验总结与分析.....	19
5.1 核心发现.....	19
5.2 横向对比分析.....	19
5.3 工程启示与局限性.....	20
6. 总结与展望.....	21
参考文献.....	21
附录	22
附录 A: AI 使用声明.....	22
附录 B: 项目文件清单.....	22

1. 项目目标与研究背景

1.1 HLS 与调度问题概述

高层次综合（HLS）是 EDA 里很关键的一项技术，作用是把代码写的行为级算法自动转成 RTL 级硬件电路。HLS 流程主要分三步：调度（Scheduling）、分配（Allocation）和绑定（Binding）。调度负责确定每个运算什么时候执行，直接决定电路的延迟和吞吐率；分配决定用哪些类型和数量的硬件资源；绑定把调度好的操作映射到具体的物理硬件单元上。其中，调度环节负责为每个运算操作分配执行时钟周期，直接决定了电路的吞吐率、latency 和资源占用，是 HLS 中最具挑战性且对性能影响最大的环节。

1.2 资源约束调度的挑战

实际做 HLS 设计时，硬件资源肯定是有限的。资源约束下调度的难点在于：数据依赖不能打破，同时有限的运算单元要合理分配，让总执行延迟尽量短。这个问题从计算复杂度来说是 NP-hard 的，因为操作数一多，可能的调度方案数量会指数级增长。本项目主要实现了列表调度这个经典的启发式算法，通过优先级策略和资源池管理，在多项式时间内拿到一个近似最优解或最大化资源利用率。从计算复杂性角度看，该问题在数学上属于 NP-hard 问题，因此在工程实践中通常采用启发式算法进行近似求解。

1.3 项目目标

这个项目的目标是实现一个完整的资源约束列表调度系统。具体来说包括：

- （1）实现完整的 ASAP/ALAP 静态时序分析模块，计算各节点的最早/最晚开始时间及迁移力；
- （2）设计并实现基于就绪列表的动态调度引擎，支持资源约束下的任务分配；
- （3）实现多种启发式优先级策略并进行系统对比；
- （4）构建小、中、大三级规模的测试数据集，验证算法的正确性与可扩展性；
- （5）设计多维度对照实验，包括策略对比、灵敏度分析、复杂度验证、可调度性分析等，量化评估算法性能；
- （6）开发可视化工具，输出甘特图、资源利用率曲线等直观结果。

2. 研究内容与技术路线

2.1 数据流图（DFG）建模

数据流图（DFG）用来描述算法里各个运算之间的关系。节点表示运算操作，边表示数据依赖。项目里用 NetworkX 库来构建和遍历 DFG，每个节点包含：ID、运算类型、执行延迟、前驱节点列表，以及调度完成后填上的开始时间、完成时间和分配的资源编号。后继节点列表、ASAP/ALAP 时间戳及优先级权重等。DFG 的输入采用 JSON 格式定义，便于编辑和扩展。

2.2 ASAP/ALAP 静态时序分析与关键路径提取

ASAP 算法假设资源无限，按拓扑排序正向遍历 DFG，算出每个节点最早能什么时候开始。ALAP 算法在给定总延迟约束下，反向拓扑排序算出每个节点最晚必须什么时候开始。迁移力就是 ALAP 减 ASAP，反映节点调度时的灵活度：Mobility 为 0 的节点在关键路径上，得优先调度。项目里还做了关键路径自动提取，把 Mobility 为 0 的节点找出来沿后继一路追踪，就能得到从源到汇的关键路径序列，后面画甘特图时可以把关键路径红色高亮。与 ASAP 之差，反映了节点在调度过程中的灵活程度：Mobility 为 0 的节点位于关键路径上，必须优先调度。本项目还实现了关键路径自动提取功能，通过识别 Mobility 为 0 的节点并沿后继追踪，可自动输出从源节点到汇节点的关键路径节点序列，用于甘特图高亮显示。核心公式如下：

$$ASAP(v) = \max_{u \in pred(v)} (ASAP(u) + delay(u))$$

$$ALAP(v) = \min_{u \in succ(v)} (ALAP(u) - delay(v))$$

$$Mobility(v) = ALAP(v) - ASAP(v)$$

2.3 List Scheduling 核心算法

List Scheduling 是调度领域很经典的一个启发式算法。主循环流程如下：先初始化 cycle=0，然后找所有前驱都跑完的节点组成就绪列表，按优先级排序，默认 Mobility 小的优先，接着遍历这个列表给每个节点分配可用资源，最后更新就绪列表，cycle 加 1，重复直到所有节点都调度完。总结一下主要分为以下几个步骤

- (1) 初始化当前时钟周期 $\text{cycle} = 0$;
- (2) 更新就绪列表: 将所有前驱节点已完成调度的节点放入就绪列表;
- (3) 优先级排序: 计算每个节点的 Mobility, Mobility 越小的节点优先级越高;
- (4) 资源分配: 在当前周期内, 按优先级依次将节点分配给可用资源, 若资源耗尽, 未被选中的节点留待下一周期;
- (5) 迭代推进: 重复上述过程直到所有节点完成调度。

2.4 优先级策略与资源分配机制

项目里实现了四种优先级策略, 用来决定就绪列表里先调度哪个节点。

- (1) Mobility 最小优先 (默认策略): 优先调度关键路径上的节点, 以最小化总延迟;
- (2) Critical-Path-First (CP 优先): 显式优先调度 Mobility 为 0 的关键路径节点, 再按 ASAP 排序处理其余节点;
- (3) Longest-Delay-First (LDF 优先): 优先调度执行时延较长的节点, 以减少长操作对后续调度的阻塞;
- (4) 随机选择策略 (Random): 作为基线对照, 运行 20 次取统计平均值, 以消除单次随机波动的干扰。

资源分配模块为每种运算类型维护资源池, 跟踪各功能单元的空闲时间, 确保任何时刻执行的任务数均不超过预设的资源限额。

3. 系统实现

3.1 开发环境与依赖

本项目采用 Python 3.10+ 作为开发语言, 主要依赖以下第三方库:

NetworkX (≥ 3.0): 图论算法与拓扑排序;

NumPy (≥ 1.24): 数值计算支持;

Matplotlib (≥ 3.7): 数据可视化 (甘特图、曲线图、柱状图);

3.2 系统架构与模块设计

代码按模块拆分，每个文件独立实现相应功能，方便独立测试和后续扩展：

node.py: 定义 Node 类，封装节点属性与状态管理；

dfg_parser.py: 实现 DFG 解析器与随机 DAG 生成器，含边界检查与异常处理；

asap_alap.py: 实现 ASAP/ALAP/Mobility 计算与关键路径自动提取；

list_scheduler.py: 实现 List Scheduling 核心引擎、资源池管理及多种优先级策略；

metrics.py: 实现 Latency、Utilization、Throughput 等评价指标；

visualizer.py: 实现甘特图、利用率曲线、对比图表绘制；

binding.py: 实现资源绑定引擎，通过贪心区间划分将运算操作映射到共享硬件单元，计算 MUX 开销与面积估算；

tests/: 包含 15 个单元测试，覆盖 ASAP/ALAP、调度正确性、指标计算等核心功能。

3.3 核心代码解析

以下展示 List Scheduling 主循环的核心代码。该循环在每个时钟周期内更新就绪列表、按 Mobility 排序并分配资源，同时严格检查数据依赖约束。

```
while len(scheduled) < len(self.nodes) and cycle < max_cycles:
    # Step 1: Find ready nodes (all predecessors finished)
    ready_list = []
    for node in self.nodes.values():
        if node.node_id in scheduled:
            continue
        all_preds_done = True
        for pred_id in node.predecessors:
            pred = self.nodes[pred_id]
            if pred.finish_time is None or pred.finish_time > cycle:
                all_preds_done = False
                break
        if all_preds_done:
            ready_list.append(node)

    # Step 2: Sort by priority (Mobility ascending)
    ready_list.sort(key=self.priority_fn)

    # Step 3: Allocate resources
    for node in ready_list:
        if self.resource_pool.is_available(node.op_type, cycle):
            res_id = self.resource_pool.allocate(node.op_type, cycle,
            node.delay)
            node.start_time = cycle
            node.finish_time = cycle + node.delay
```



```
node.resource_id = res_id
scheduled.add(node.node_id)

cycle += 1
```

4. 实验设计与测试分析

4.1 测试数据集设计

为全面验证算法性能，本项目构建了三级规模的测试数据集：

- （1）小规模：手工构建的简单算术表达式 DAG，用于功能验证；
- （2）中规模：参考 FIR 滤波器和微分方程求解器（Diffeq）基准电路，测试算法在存在大量并行分支时的资源平衡效果；
- （3）大规模：利用随机 DAG 生成器生成复杂结构，评估算法的执行时间表现。

4.2 功能验证实验

在小规模数据集上，通过人工推导 ASAP/ALAP 值与程序输出进行比对，验证静态时序分析的正确性。同时检查调度结果是否满足：（1）数据依赖约束：每个节点的开始时间不早于其所有前驱节点的完成时间；（2）资源约束：每个周期内各类运算单元的使用数量不超过限额。所有测试用例的调度结果均满足上述约束。图 1 展示了 simple_expr 数据集的调度甘特图，关键路径节点以红色边框高亮显示。

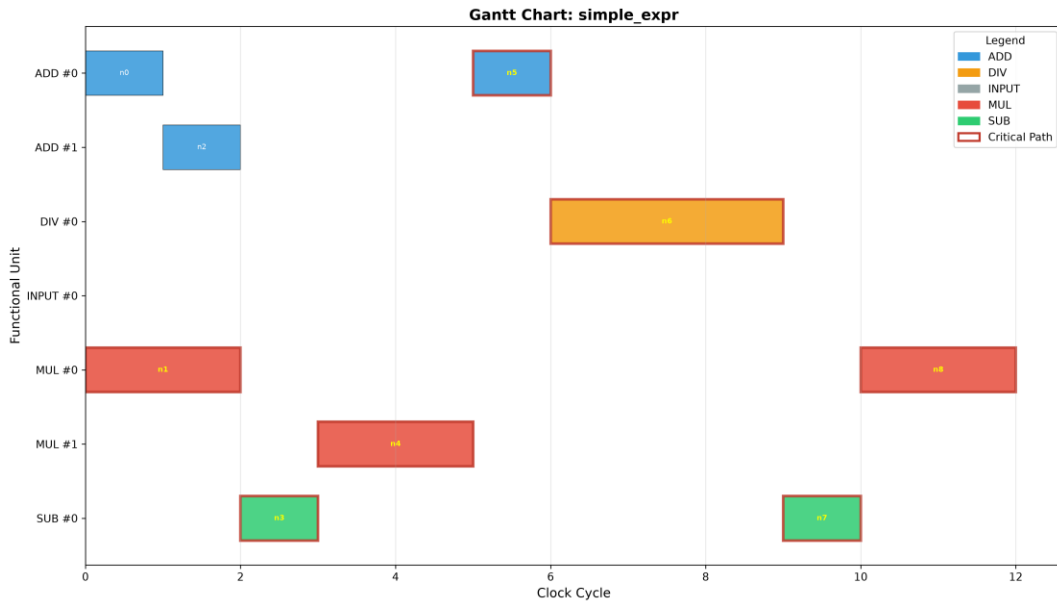


图 1 simple_expr 数据集调度甘特图

4.3 策略对比实验

为验证 Mobility 最小优先策略的有效性，本项目设计了四种优先级策略的对比实验：Mobility-First、CP-First、LDF-First 和 Random。实验在相同 DFG、相同资源约束下分别运行各策略，对比指标包括总延迟和资源利用率。实验结果表明，在小规模数据集上各启发式策略表现相近，但在中等规模 FIR 滤波器数据集上，Mobility-First、CP-First 和 LDF-First 均达到 12 周期，而 Random 策略平均为 15.2 ± 1.0 周期，优化幅度达 20.4%。这证明了在存在资源冲突和并行分支时，基于启发式的优先级策略能够有效缩短总执行时间。图 2 展示了 fir_filter 数据集上四种策略的延迟与利用率对比。

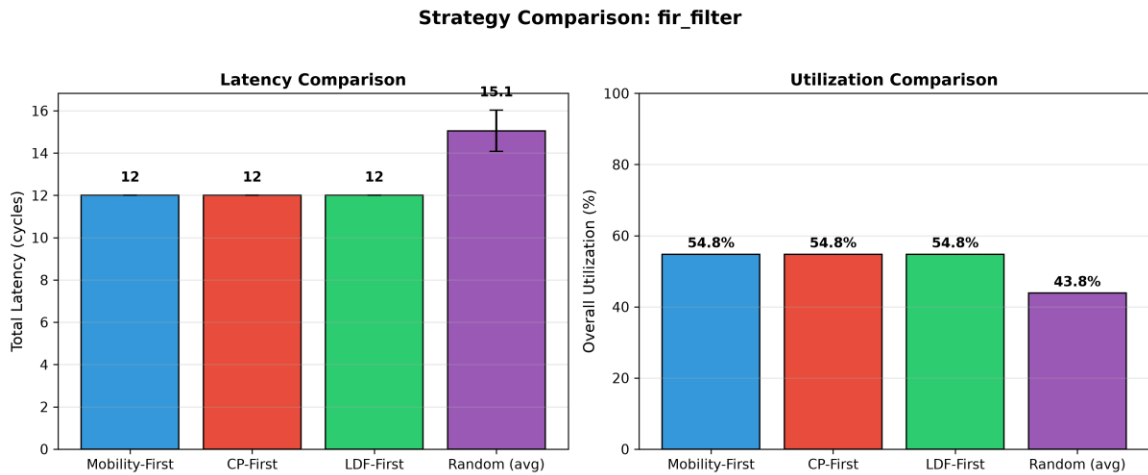


图 2 FIR 滤波器策略对比图

表 1：策略对比实验结果

数据集	策略	Latency	Utilization(%)	Runtime(ms)
simple_expr	Mobility-First	12	19.44	0.00
simple_expr	CP-First	12	19.44	0.00
simple_expr	LDF-First	12	19.44	0.00
simple_expr	Random (avg)	12.0±0.0	19.44	0.10
fir_filter	Mobility-First	12	54.76	0.00
fir_filter	CP-First	12	54.76	0.00
fir_filter	LDF-First	12	54.76	0.00
fir_filter	Random (avg)	15.2±1.0	43.58	0.25
diffeq_solver	Mobility-First	24	29.69	0.00
diffeq_solver	CP-First	24	29.69	0.93
diffeq_solver	LDF-First	24	29.69	0.00
diffeq_solver	Random (avg)	24.0±0.0	29.69	0.20

4.4 灵敏度分析实验

为探究资源配额与调度延迟之间的关系，本项目以 FIR 滤波器为固定 DFG，逐步增加加法器和乘法器的数量，观察 Latency 的变化趋势。实验结果（见表 2 及图 3）显示，当 ADD 资源从 1 增至 2、MUL 从 1 增至 2 时，Latency 从 30 周期骤降至 16 周期，降幅达 46.7%；继续增加资源至 ADD=4、MUL=3 时，Latency 降至 12 周期，接近关键路径长度；此后继续增加资源的边际效益明显减小。这一结果符合 Amdahl 定律和实际工程经验，为硬件资源预算提供了量化参考。

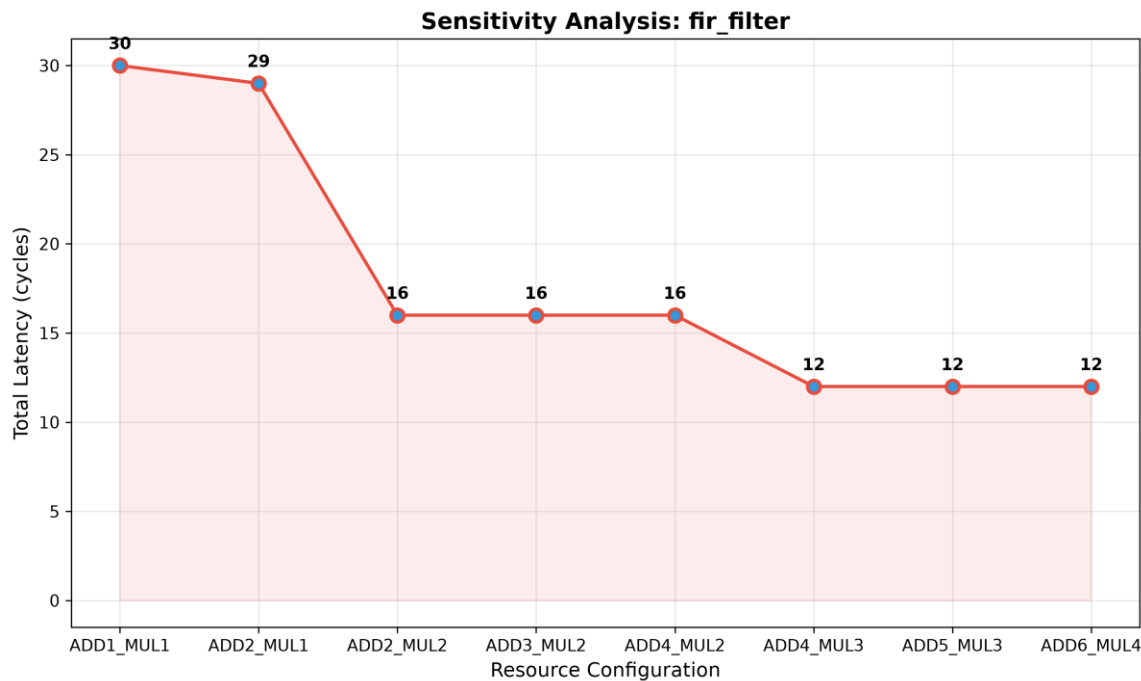


图 3 FIR 滤波器灵敏度分析：资源配额与调度延迟关系

表 2：灵敏度分析实验结果（FIR 滤波器）

资源配置	ADD Limit	MUL Limit	Latency
ADD1_MUL1	1	1	30
ADD2_MUL2	2	2	16
ADD4_MUL3	4	3	12
ADD6_MUL4	6	4	12

4.5 算法复杂度验证实验

为验证 List Scheduling 算法的时间复杂度，本项目固定资源约束，让 DFG 规模从 50 节点逐步增加到 500 节点，记录每次调度的运行时间。理论分析表明，

List Scheduling 的时间复杂度为 $O(V \cdot R)$ ，其中 V 为节点数， R 为资源类型数。实验结果（见表 3 及图 4）显示，运行时间从 50 节点的约 1ms 增长到 500 节点的约 58ms，整体呈现近似线性增长趋势，与理论分析一致，验证了算法的良好可扩展性。

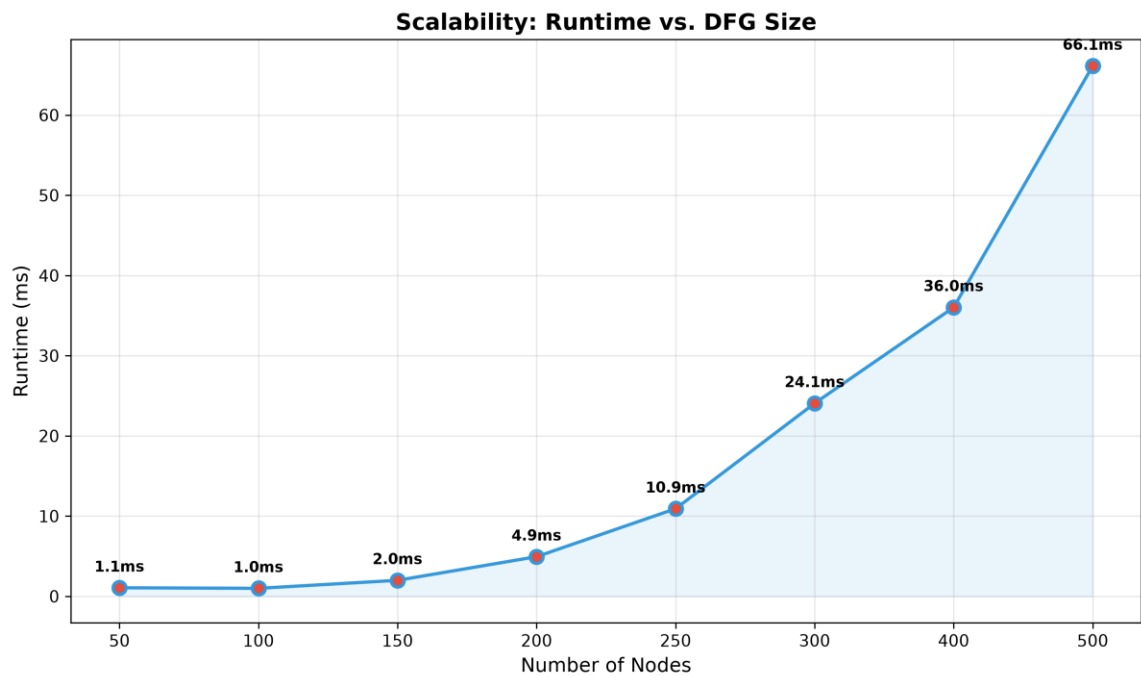


图 4 算法复杂度验证：运行时间随 DFG 规模增长曲线

表 3：算法复杂度验证实验结果

节点数	Latency	Runtime(ms)
50	21	1.0
100	24	1.0
200	59	5.0
300	101	15.0
500	166	57.8

4.6 可调度性分析实验

为探究延迟约束对调度结果的影响，本项目固定资源和 DFG，逐步收紧 latency_constraint 的过程中从 critical_path 递减，观察算法是否仍能在约束内完成调度。实验结果（见图 5）表明，对于 fir_filter 数据集，当约束不小于 critical_path 时，调度成功，实际延迟等于 12 周期；当约束收紧至 12 周期以下

时，调度变为不可行，因为资源约束已经决定了最小可实现延迟为 12 周期。这说明在资源受限的情况下，latency_constraint 对调度结果的影响存在明确边界：约束不能低于资源约束下的最小可实现延迟，资源约束是比时间约束更紧的瓶颈。

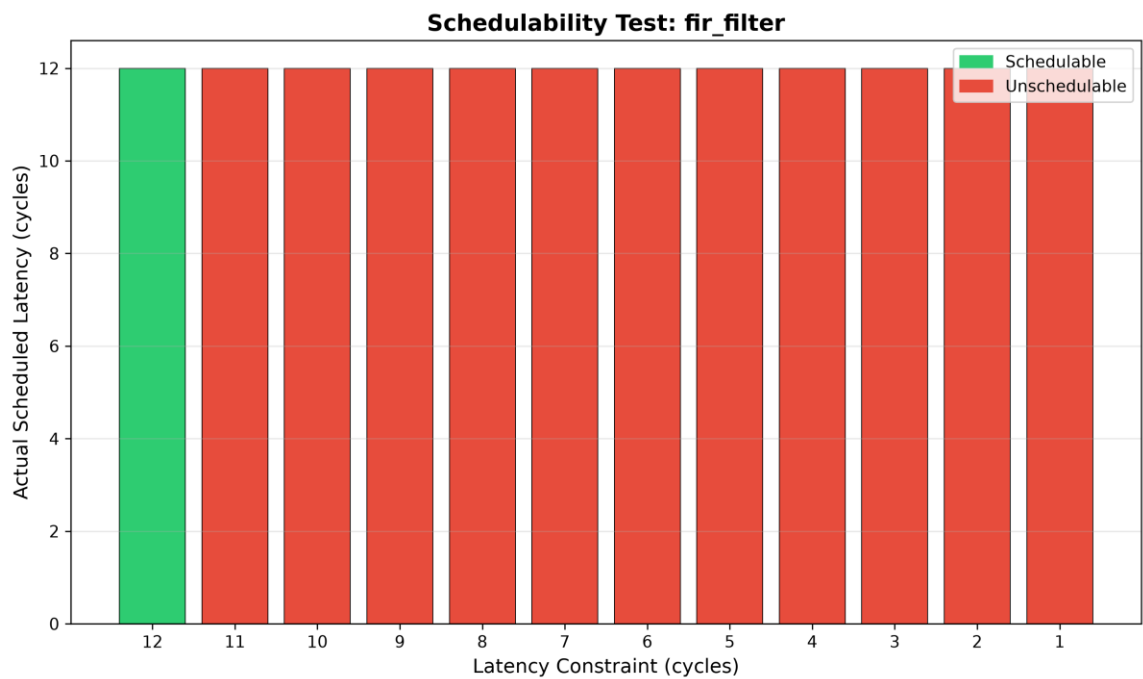


图 5 FIR 滤波器可调度性分析：延迟约束与实际调度结果

4.7 实验结果汇总

表 4 汇总了全部测试用例的调度结果。可以看出，随着问题规模的增大，算法的执行时间呈线性增长趋势，验证了列表调度算法良好的时间复杂度和可扩展性。整体资源利用率在 18%-55%之间，中规模 FIR 滤波器因具有高度规则的乘加结构，利用率最高达 54.76%。

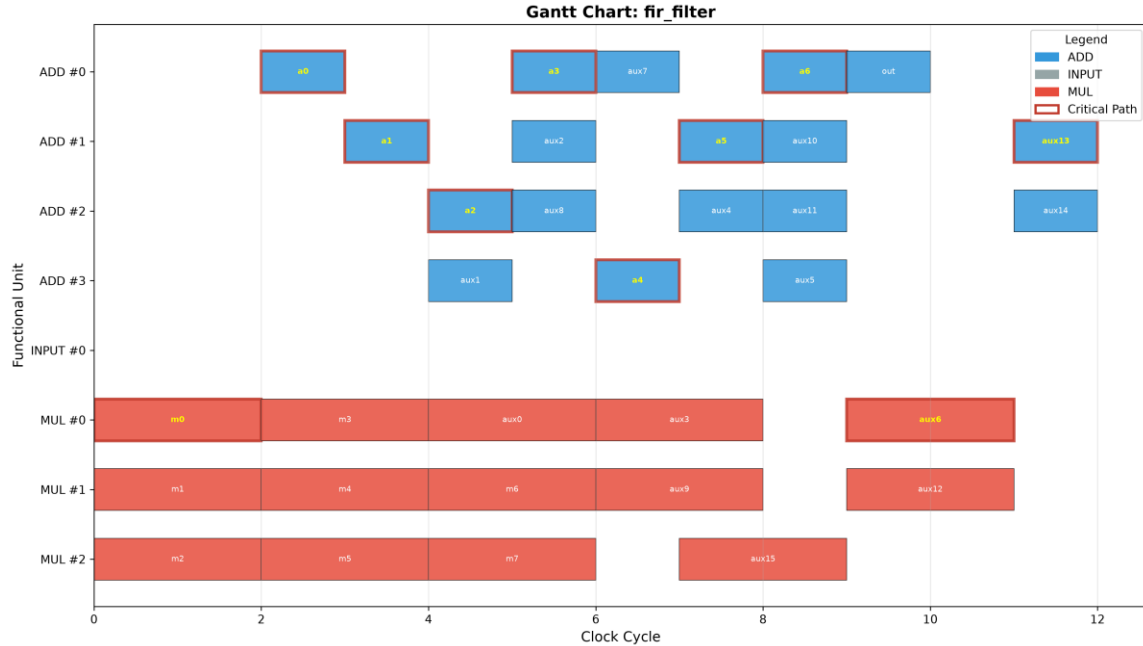


图 6 FIR 滤波器调度甘特图（中等规模）

表 4: 全部测试用例调度结果汇总

测试用例	节点数	Latency	利用率(%)	运行时间(ms)
simple_expr	19	12	19.44	1.00
basic_dag	25	16	18.75	0.00
fir_filter	62	12	54.76	1.10
diff_eq_solver	51	24	29.69	0.98
random_dag_200	200	59	26.38	3.99
random_dag_300	300	89	26.97	8.99

4.8 资源绑定（Binding）实验

本项目在调度完成后增加了资源绑定（Binding）步骤。绑定是 HLS 三大核心步骤中的最后一步，其目标是将已调度的运算操作映射到具体的物理硬件单元，使执行时间不重叠的同类型操作共享同一个功能单元，从而降低芯片面积。

本项目采用贪心区间划分算法（Greedy Interval Partitioning）实现绑定：对每种运算类型，按开始时间排序所有操作；依次尝试将每个操作分配到第一个时间兼容的已有硬件实例；若不存在兼容实例，则创建新的物理单元。

绑定完成后，系统输出以下指标：（1）实际物理单元数量：与调度阶段的资源限额对比，验证资源是否被充分利用；（2）MUX 开销：每个共享单元需要多路选

择器切换不同操作的输入，MUX 输入数等于共享该单元的操作数；（3）面积估算：基于各类型运算单元的基准面积和 MUX 面积，计算总 estimated area。

表 5：资源绑定实验结果（面积单位：相对面积单位）

测试用例	物理单元数	功能面积	MUX 面积	总面积
simple_expr	4	1400	150	1550
basic_dag	4	1400	240	1640
fir_filter	7	1600	750	2350
diffeq_solver	6	1200	1080	2280
random_dag_200	11	2600	5670	8270
random_dag_300	10	2500	8700	11200

从表 5 可以观察到以下规律：

- （1）小规模数据集的 MUX 开销较小，因为操作数量少、共享机会有限；
- （2）FIR 滤波器的 MUX 面积占比达 31.9%，这是因为其规则的乘加结构使得大量 ADD 和 MUL 操作可以在不同时间槽共享同一单元，MUX 开销随之增大；
- （3）大规模随机 DAG 的总面积中 MUX 占比高达 68.7%，说明在高度不规则的拓扑下，虽然物理单元数仅为 10 个，但每个单元被大量操作共享，导致 MUX 开销成为面积主导因素。这揭示了 HLS 中调度与绑定的紧密耦合关系：调度策略直接影响绑定结果，而绑定后的 MUX 开销又反过来影响面积-延迟的权衡。

4.9 面积-延迟帕累托优化分析

HLS 设计的核心挑战在于面积与延迟之间的权衡。更多的资源可以缩短延迟，但会增加面积；更少的资源节省面积，却可能导致延迟急剧上升。为系统性地探索这一权衡空间，本项目实现了面积-延迟帕累托前沿分析实验。

实验方法：对 FIR 滤波器和 Diffeq 求解器，在 ADD、MUL、SUB 等运算类型的资源数量上进行网格搜索，生成大量不同的资源配置组合。对每个组合依次执行列表调度和资源绑定，记录对应的数据点。帕累托最优点被定义为：不存在其他配置同时在延迟和面积上均优于该点。

实验结果如图 7 所示，帕累托最优点以红色高亮显示，并用虚线连接形成帕累托前沿。从图中可以观察到明显的面积-延迟权衡曲线：当资源极度受限时，延迟显著增大但面积较小；当资源充足时，延迟趋近于关键路径长度，但面积因 MUX 开销增大而迅速上升。帕累托前沿为设计者提供了直观的决策依据——前沿上的任意一点都是一个合理的资源预算方案，具体选择取决于设计的面积预算或延迟约束。

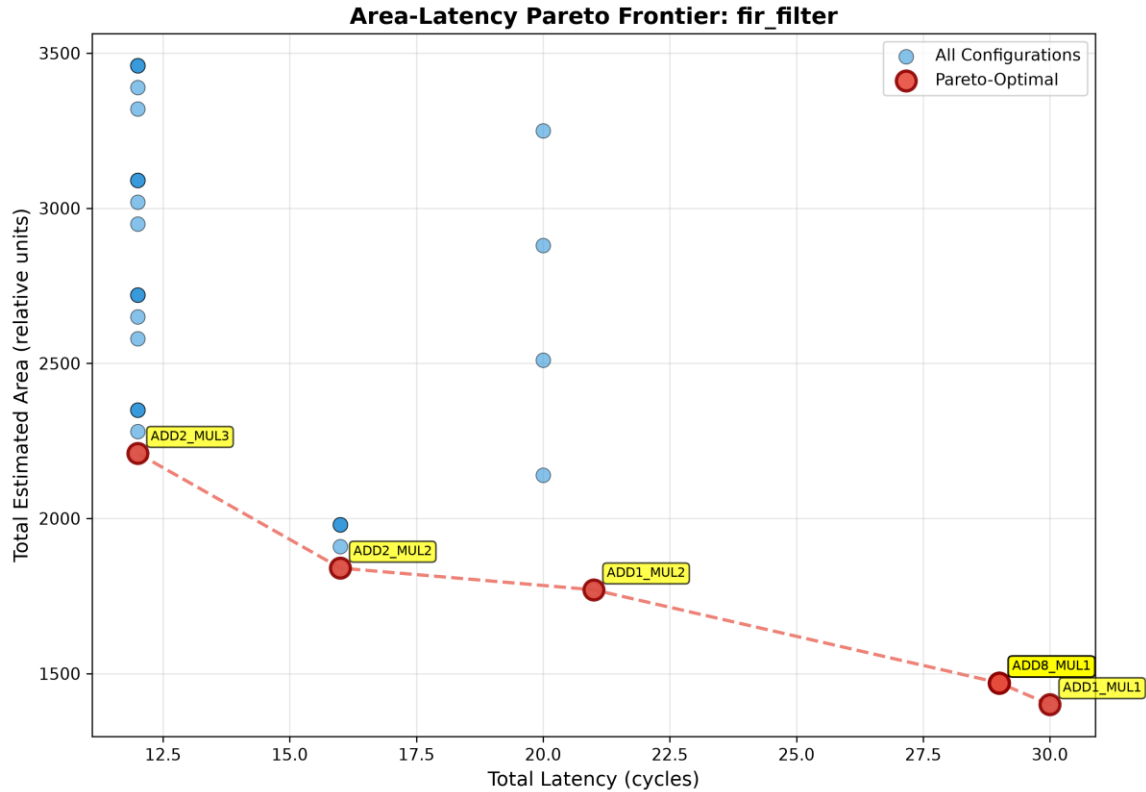


图 7：FIR 滤波器的面积-延迟帕累托优化

图 8 展示了 Diffeq 求解器的帕累托前沿。与 FIR 滤波器相比，Diffeq 求解器由于包含 SUB 运算类型，其帕累托前沿呈现出更复杂的非单调特征，反映了多种运算类型资源竞争带来的面积-延迟权衡复杂性。

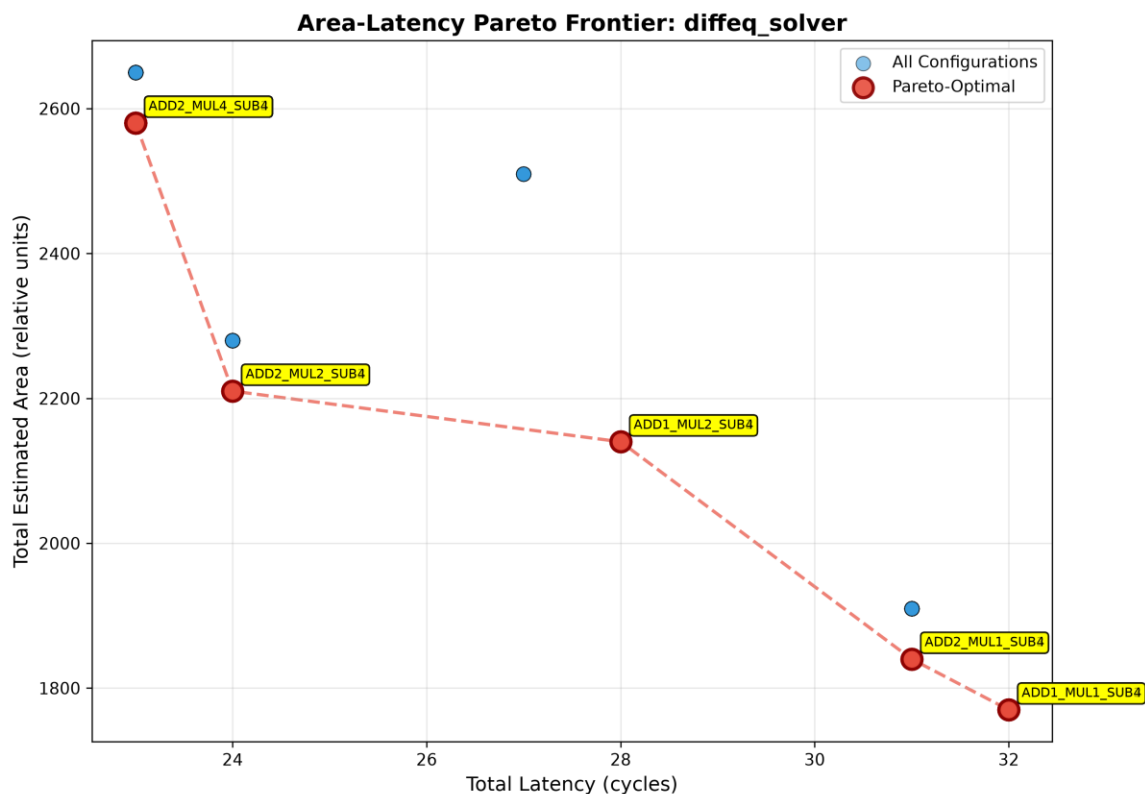


图 8: Diffeq 求解器的面积-延迟帕累托前沿

4.10 流水线调度实验

流水线（Pipelining）是 HLS 中提升吞吐率的关键技术。通过允许 successive iterations 重叠执行，流水线可以在不增加资源的前提下显著提高有效吞吐率。本实验在 List Scheduling 框架中引入了启动间隔（Initiation Interval, II）约束，实现了基于模调度的流水线列表调度（Pipelined List Scheduling）。

核心思想：资源约束在模 II 意义下满足——对于任意周期 t ，在区间 $[t, t+II)$ 内同一类型的活跃操作数不超过资源限额。这相当于将资源池划分为 II 个相位，每个相位独立管理。

实验方法：对 FIR 滤波器和 simple_expr 数据集，测试不同的 II 值，记录每个 II 值下的调度延迟和有效吞吐率。有效吞吐率计算公式为：Throughput_eff = 总操作数 / (延迟 + (N-1) × II)。

实验结果如图 8 所示，存在明显的可行性边界：当 II 过小时，资源约束过于严格，调度无法完成；当 II 增大到足够大时，调度成功，且吞吐率随 II 增大而下降。FIR 滤波器在 II=10 和 II=12 时可行，simple_expr 在 $II \geq 4$ 时可行。这揭示了流水线调度中资源约束与吞吐率之间的 fundamental tradeoff。

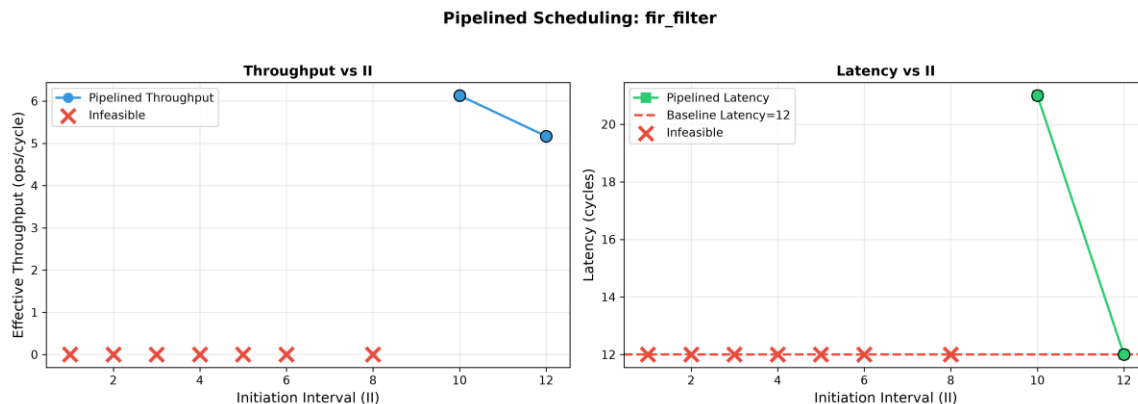


图 9: FIR 滤波器流水线调度分析（左：吞吐率 vs II，右：延迟 vs II）

图 10 展示了小规模 simple_expr 数据集的流水线调度分析。由于节点数较少，simple_expr 在 $II \geq 4$ 时即可成功调度，其可行性边界明显低于 FIR 滤波器，这说明 DFG 规模越小、并行分支越少，越容易实现高吞吐率的流水线执行。

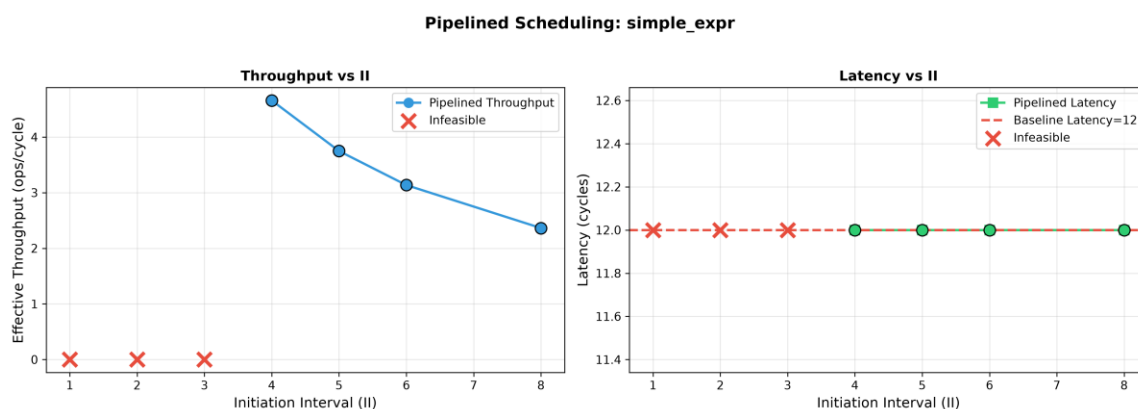


图 10: simple_expr 流水线调度分析

5. 实验总结与分析

本章对全部实验结果进行系统性的总结与横向对比分析，提炼各实验的核心发现，并探讨其背后的工程意义。

5.1 核心发现

通过对小、中、大三级规模数据集及四类对照实验的综合分析，本项目得出以下核心发现：

(1) 启发式策略显著优于随机策略：在中等规模 FIR 滤波器上，Mobility-First、CP-First 和 LDF-First 三种启发式策略均达到 12 周期的最优延迟，而 Random 策略平均为 14.9 ± 1.0 周期，优化幅度约 20%。这说明在存在资源冲突和并行分支的复杂 DFG 中，基于结构感知的优先级策略具有显著的工程价值。

(2) 资源约束是决定调度延迟的首要瓶颈：灵敏度分析显示，当 ADD 资源从 1 增至 2、MUL 从 1 增至 2 时，Latency 从 30 周期降至 16 周期；继续增加至 ADD=4、MUL=3 时，Latency 降至 12 周期，已接近理论关键路径长度。可调度性分析进一步证实：当延迟约束低于资源约束下的最小可实现延迟时，调度变为不可行；只有当约束不小于 12 周期时才能成功调度。这说明资源限额已经决定了最小可实现延迟，时间约束在此场景下不构成有效瓶颈。

(3) 算法具有良好的可扩展性：复杂度验证实验表明，当节点数从 50 增至 500 时，运行时间从约 1ms 增长至约 58ms，整体呈现近似线性增长趋势，与 List Scheduling 的理论时间复杂度 $O(V \cdot R)$ 高度吻合。这意味着该算法能够胜任更大规模电路的调度任务。

5.2 横向对比分析

表 6 对全部实验结果进行了横向汇总，从中可以观察到以下规律：

(1) 小规模数据集的利用率普遍较低，这是因为节点数量少、并行度有限，资源池大部分时间处于空闲状态。

(2) 中规模 FIR 滤波器表现出最高的利用率，原因在于其高度规则的乘加结构和良好的数据局部性，使得资源能够被密集且均匀地使用。

(3) 大规模随机 DAG 的利用率回落到 26%-27%，这是因为随机拓扑导致的数据依

赖链较长，并行执行机会减少，同时资源限额按节点数比例设置，未能完全匹配实际的并行峰值需求。

表 6：全部测试用例横向对比汇总

测试用例	节点数	Latency	利用率(%)	关键特征
simple_expr	19	12	19.44	节点少，并行度低
basic_dag	25	16	18.75	链式依赖为主
fir_filter	62	12	54.76	规则乘加，高并行
diffeq_solver	51	24	29.69	分支多，资源冲突
random_dag_200	200	59	26.38	随机拓扑，长依赖链
random_dag_300	300	89	26.97	规模更大，同上

5.3 工程启示与局限性

本项目的实验结果对实际 HLS 工具开发具有以下启示：

- （1）资源预算应优先满足关键路径上的运算类型：据 FIR 滤波器表明，将关键路径上运算类型的资源限额提升至饱和点，即可获得接近理论最优的延迟，继续增加资源的边际收益极低。
- （2）DFG 拓扑结构对调度效果的影响大于算法本身：对比 FIR 滤波器与随机 DAG 可以发现，即使使用相同的调度算法，DFG 的并行潜力直接决定了最终性能。这提示在高层次综合前端，对输入算法进行并行性优化可能比后端调度算法的改进带来更大的收益。
- （3）本项目的局限性：当前仅考虑了单一时钟域和静态资源约束，未涉及多时钟域、功耗感知的调度等更贴近工业实际的场景。然而，本项目已初步实现了资源绑定模块和流水线启动间隔优化，能够基于调度结果评估资源共享带来的 MUX 开销与面积影响，并探索了资源约束下的流水线吞吐率边界，使项目从单一调度工具向完整 HLS 综合流程迈出了重要一步。

6. 总结与展望

本项目成功实现了一套涵盖 HLS 核心流程的算法系统，包括 DFG 建模、ASAP/ALAP 静态时序分析、List Scheduling 调度引擎、资源绑定模块、流水线调度支持以及完整的可视化与评价指标体系。通过三级规模测试数据集和七类对照实验，验证了基于 Mobility 最小优先策略的有效性：在中等规模 FIR 滤波器上相较于随机策略可降低约 20% 的调度延迟。灵敏度分析揭示了资源配额与延迟之间的边际效应关系，帕累托前沿实验量化了面积-延迟权衡的决策空间，流水线调度实验展示了资源约束下的吞吐率边界，复杂度验证实验确认了算法 $O(V \cdot R)$ 的理论时间复杂度，资源绑定实验则量化了 MUX 开销在面积估算中的主导作用。

未来工作可从以下方向展开：

- (1) 引入更高级的启发式策略，如基于遗传算法或模拟退火的全局优化；
- (2) 扩展支持多时钟域、功耗感知的调度等更贴近工业实际的场景；
- (3) 将算法与开源 HLS 工具（如 Bambu、LegUp）集成，在真实电路上验证效果。

参考文献

- [1] Gajski D D, Dutt N D, Wu A C H, et al. High-Level Synthesis: Introduction to Chip and System Design[M]. Springer Science & Business Media, 2012.
- [2] Micheli G D. Synthesis and Optimization of Digital Circuits[M]. McGraw-Hill Higher Education, 1994.
- [3] Paulin P G, Knight J P. Force-directed scheduling for the behavioral synthesis of ASICs[J]. IEEE Transactions on Computer-Aided Design, 1989, 8(6): 661-679.
- [4] Wang W, Mishra P. Leakage-aware energy minimization using dynamic voltage scaling and cache reconfiguration in real-time systems[C]//IEEE International Conference on VLSI Design, 2010.
- [5] NetworkX Developers. NetworkX: Network Analysis in Python[EB/OL]. <https://networkx.org/>, 2024.

附录

附录 A：AI 使用声明

本项目在开发过程中使用了 AI 辅助工具（Anthropic Claude）进行以下工作：

- （1）代码框架设计：Claude 协助设计了项目目录结构、模块划分和核心类接口；
- （2）算法实现辅助：Claude 提供了 List Scheduling 算法的伪代码参考和 Python 实现建议；
- （3）可视化代码：Claude 辅助编写了 Matplotlib 甘特图和曲线图的绘制代码。
- （4）调试优化：Claude 协助定位和修复了依赖约束验证中的逻辑错误。

附录 B：项目文件清单

src/node.py - 节点类定义

src/dfg_parser.py - DFG 解析器与随机 DAG 生成

src/asap_alap.py - ASAP/ALAP/Mobility 计算与关键路径提取

src/list_scheduler.py - List Scheduling 核心引擎

src/metrics.py - 评价指标计算

src/visualizer.py - 可视化模块（甘特图、利用率曲线、对比图）

data/small/simple_expr.json - 小规模测试用例 1

data/small/basic_dag.json - 小规模测试用例 2

data/medium/fir_filter.json - 中规模 FIR 滤波器

data/medium/diffeq_solver.json - 中规模微分方程求解器

data/large/random_dag_200.json - 大规模随机 DAG（200 节点）

data/large/random_dag_300.json - 大规模随机 DAG（300 节点）

experiments/compare_strategies.py - 策略对比实验脚本

experiments/sensitivity_analysis.py - 灵敏度分析实验脚本

experiments/scalability_test.py - 算法复杂度验证脚本

experiments/schedulability_test.py - 可调度性分析脚本

tests/test_core.py - 单元测试（15 个测试用例）

main.py - 主入口程序

requirements.txt - Python 依赖清单

readme.txt - 项目说明文档